2 TIER ARCHITECTURE PROJECT

VINEETH KUMAR M

INDEX PAGE

1. INTRODUCTION
2. ARCHITECTURE
3. WORKFLOW
4. EXPLANATION
5. RESULT

1. INTRODUCTION

This project is a cloud-based student form creation developed using AWS services. It follows a multi-tier architecture where frontend, backend, and database are deployed separately.

2. ARCHITECTURE:

a. Frontend – ec2 server (ubuntu) using html.
b. Backend – flask api (python)

3. WORKFLOW:

a. Create frontend EC2 server
b. Create VPC for EC2
c. Create backend EC2 server
d. Create a mysql DB
e. Install nginx server on frontend
f. Install python on backend

4 – EXPLANATION:

Step-1 Create a VPC

Step-2 Create 2 subnets,

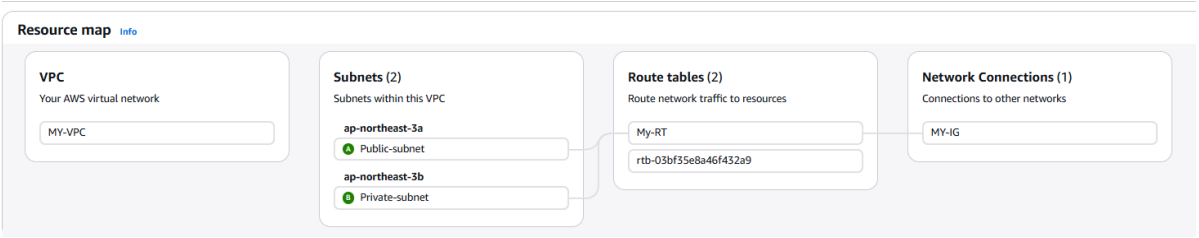
- Create 1 public subnet – frontend.
- Create 1 private subnet – Backend

Step-3 Create Route table,

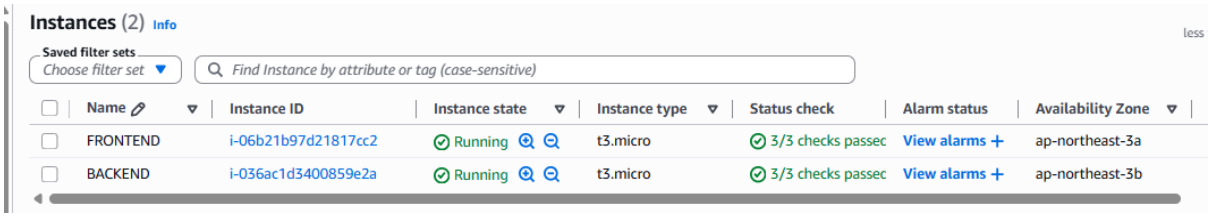
- Create a Public RT.
- Create a Private RT.

Step-4 Create internet gateway then,

- Attach the IGW to VPC.
- Attach the public subnet to the public Route table.
- Attach the private subnet to the public Route table.
- Edit subnet association explicit connection.

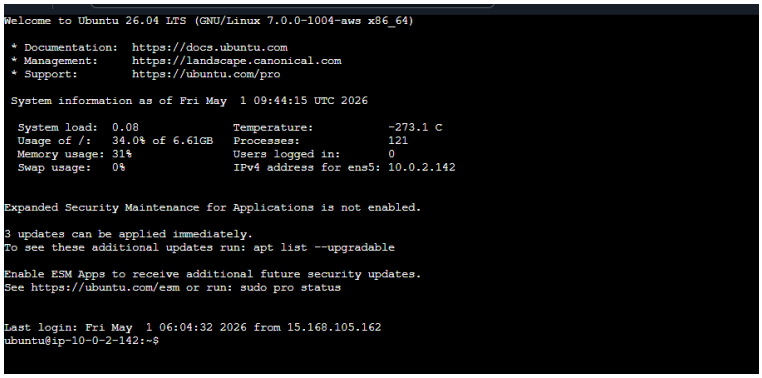


Step -5 Create a EC2 INSTANCE



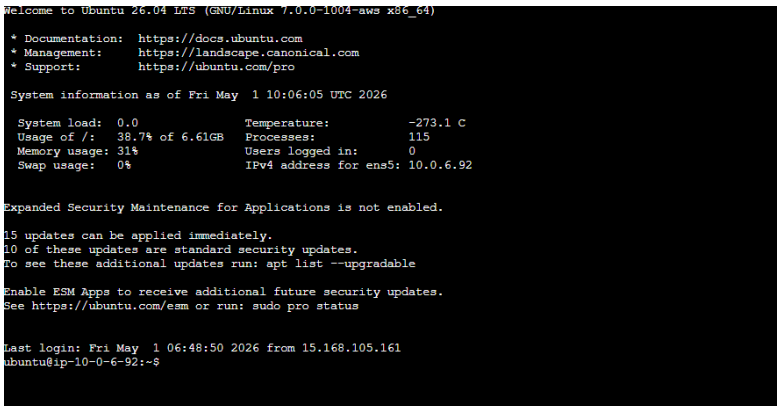
Step-6 Launch front-end EC2 instance

- Frontend – Nginx.
- Operating system: Ubuntu AMI
- Subnet: Public
- Attached the SG group
- Auto assign IP.



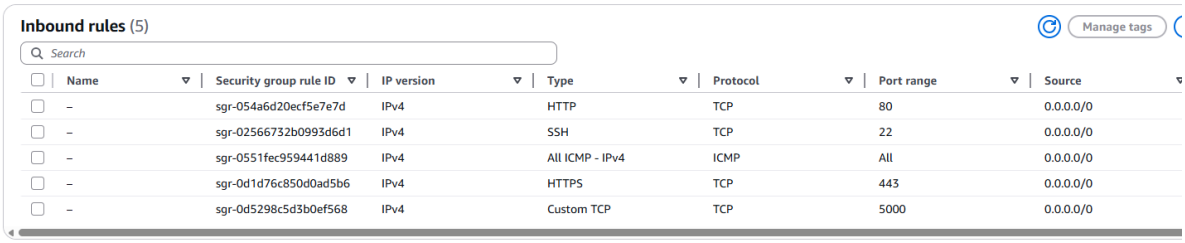
Step-6 Launch Back-end EC2 instance.

- Front end – Flash API
- Operating system: Ubuntu AMI
- Subnet: Private
- Attached the SG group
- Auto assign IP.



Step-7 Go-to Security group inbound rules

- Allow the ports which you need.
- Allow custom port 5000 (for python)



Connect your server and use below some commands to update your kernel and install nginx,

LINUX – OPERATING SYSTEM

- sudo apt update –y

```
ubuntu@ip-10-0-6-92:~$ sudo apt update -y
```

- sudo nano “key pair name” (update your key pair name)

```
ubuntu@ip-10-0-2-142:~$
ubuntu@ip-10-0-2-142:~$ sudo nano "key pair name"
```

- sudo chmod 400 “key pair name” (change the permission to your key pair)

```
ubuntu@ip-10-0-6-92:~$
ubuntu@ip-10-0-6-92:~$ sudo chmod 400 "key pair name"
```

NGINX - SERVER

Install nginx server in frontend by using below commands:

- sudo apt install nginx
- default path (nginx) var/www/html/index.nginx-debian.html
- update your html code in that .html page
- now your front-end is working

Create a connection between FRONT-END TO BACKEND

FRONT-END ============== API ==============BACK-END

BACKEND - SERVER

Connect your server and use below some commands to install nginx and update your kernel,

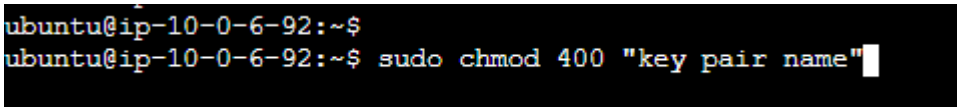
- sudo apt update –y

```
ubuntu@ip-10-0-6-92:~$ sudo apt update -y
```

- sudo nano “key pair name” (update your key pair name)

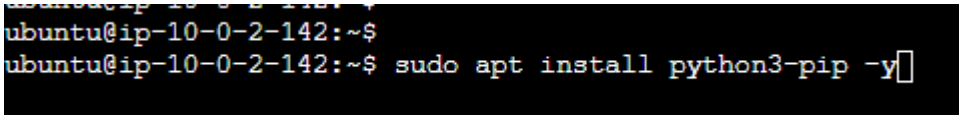
```
ubuntu@ip-10-0-2-142:~$
ubuntu@ip-10-0-2-142:~$ sudo nano "key pair name"
```

- sudo chmod 400 “key pair name” (change the permission to key pair)

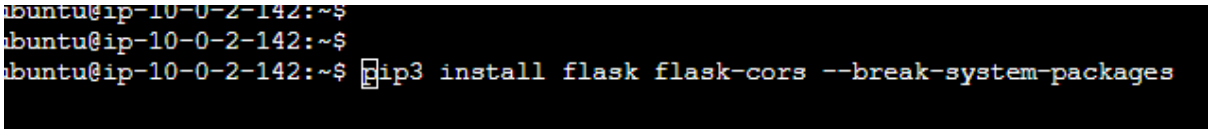


You need to install python in you back-end-server by using below commands,

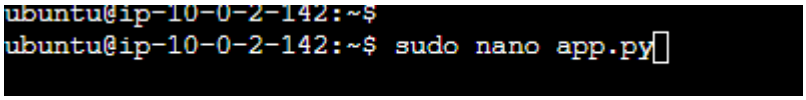
- sudo apt install python3-pip-y



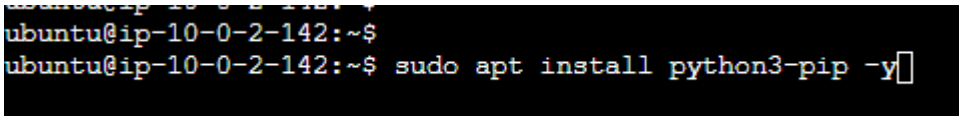
- pip3 install flask flask-cors --break-system-packages



- create app.py (update your flask code)



- python3 app.py (now python is running on your server)



Use the below html code in front-end-server

- go to your nginx default path and,
- edit the .html file with below html

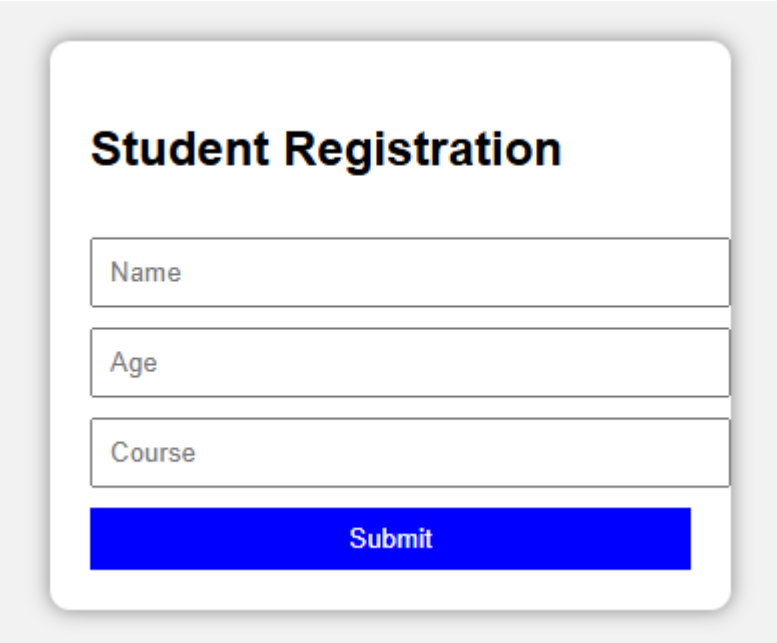
```
<!DOCTYPE html>
        <!DOCTYPE html>
<html>
<head>
  <title>Student Registration</title>
  <style>
    body {
      font-family: Arial;
      background: #f2f2f2;
      padding: 30px;
    }
```

```
    .box {
      width: 320px;
      background: white;
      padding: 20px;
      border-radius: 10px;
      box-shadow: 0 0 10px gray;
    }

    input, button {
      width: 100%;
      margin-top: 10px;
      padding: 8px;
    }

    button {
      background: blue;
      color: white;
      border: none;
      cursor: pointer;
    }

    #list {
      margin-top: 20px;
      padding: 10px;
      background: #e6ffe6;
      border-radius: 5px;
    }
  </style>
</head>
<body>

<div class="box">
  <h2> Student Form</h2>

  <form id="form">
    <input type="text" id="name" placeholder="Name" required>
    <input type="number" id="age" placeholder="Age" required>
    <input type="text" id="course" placeholder="Course" required>

    <button type="submit">Submit</button>
```

```
  </form>

  <div id="list"></div>
</div>

<script>
const listDiv = document.getElementById("list");

document.getElementById("form").addEventListener("submit", function(e) {
  e.preventDefault();

  const data = {
    name: document.getElementById("name").value,
    age: document.getElementById("age").value,
    course: document.getElementById("course").value
  };

  fetch("http://YOUR_IP_ADDRESS:5000/student", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(data)
  })
  .then(res => res.json())
  .then(response => {
    listDiv.innerHTML = "<h3>Student List:</h3>";

    response.students.forEach((s, index) => {
      listDiv.innerHTML += `
        <p>${index + 1}. ${s.name} - Age: ${s.age} - Course: ${s.course}</p>
      `;
    });
  })
  .catch(error => {
    listDiv.innerHTML = "❌ Server Error!";
    console.error(error);
  });
});
</script>
```

</body>
</html>

- Try your Frontend ip address in your chrome.

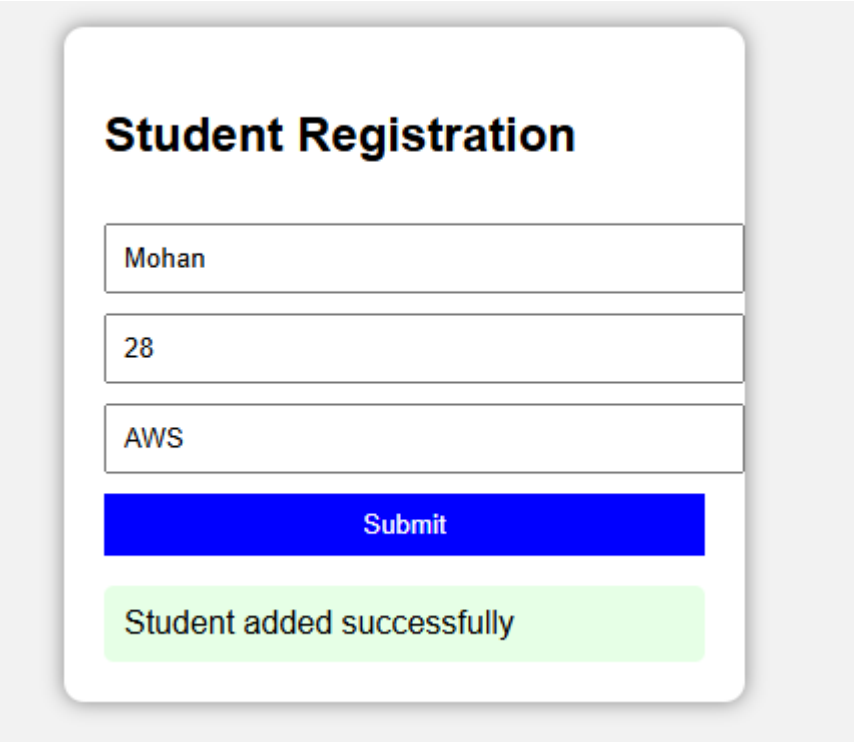


- Go to backend server and update your app.py by using below code which you created,

Update your app.py (flask code) by below code: **Just copy paste**

```
from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

students = []  # store all students

@app.route('/student', methods=['POST'])
def add_student():
  data = request.get_json()

  student = {
    "name": data.get('name'),
    "age": data.get('age'),
    "course": data.get('course')
  }
```

```
    students.append(student)

    return jsonify({
        "message": "Student added successfully",
        "students": students
    })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```



THANK YOU